

MS579 Computer Vision

Assignment 4.1

Collecting and Labeling Data

Edge Impulse Object Detection: J2534 VCI and Key Fob Dataset

Student: Harold L. R. Watkins

Course: MS579 — Computer Vision

Institution: University of Advancing Technology (UAT)

Submission Date: June 2026

GitHub Repository: <https://github.com/LRTechpro/MS579-Edge-Impulse-J2534-KeyFob-Detection>

1. Introduction and Process Narrative

This report documents the complete data collection and labeling pipeline developed for MS579 Assignment 4.1, fulfilling the requirement to construct, label, and deploy a custom object detection model using Edge Impulse. The project targeted two automotive diagnostic objects commonly encountered in vehicle cybersecurity and ADAS diagnostic workflows: a J2534 Vehicle Communication Interface (VCI) adapter and a remote key fob. These artifacts represent real-world automotive security tools, making them directly applicable to the practitioner's professional domain.

The process followed a systematic pipeline: (1) hardware configuration of an ESP32-CAM module for autonomous image acquisition, (2) structured data collection using the EloquentEsp32cam library's HTTP image collection server, (3) dataset curation and organization, (4) upload and labeling in Edge Impulse using manual bounding box annotation, (5) impulse design with FOMO (Faster Objects, More Objects) MobileNetV2 architecture, (6) model training, validation, and testing, and (7) deployment as an Arduino-compatible library for embedded inference on the ESP32-CAM.

The final dataset consisted of 180 images distributed across three classes: 75 key fob images (label: **keyfob**), 75 J2534 VCI images (label: **J2534VCI**), and 30 background images with no labeled objects. Edge Impulse automatically partitioned the dataset into a 79%/21% train/test split (141 training samples, 39 test samples). All training images were manually annotated with tight bounding boxes to maximize annotation quality and model precision.

2. Selected Artifacts

Two automotive diagnostic artifacts were selected as detection targets for this project. These objects were chosen based on their relevance to automotive cybersecurity workflows and their physical distinctiveness, which facilitates reliable visual discrimination by a constrained FOMO model.

2.1 J2534 Vehicle Communication Interface (VCI)

The J2534 VCI is a standardized pass-through interface defined by SAE J2534, enabling communication between a laptop or diagnostic host and a vehicle's OBD-II port across multiple ECU communication protocols (CAN, ISO 9141-2, KW2000, J1850 PWM/VPW). In automotive cybersecurity, the J2534 adapter represents a

high-value attack surface: an adversary with physical access to this device can reflash ECU firmware, extract calibration data, or replay diagnostic sequences. Training a visual recognition model on this artifact supports automated anomaly detection in secure facility workflows.

2.2 Remote Key Fob

The remote key fob is a short-range RF transmitter used for passive entry and passive start (PEPS) systems. From a security standpoint, key fobs are targets for relay attacks, signal amplification attacks, and clone-and-replay attacks. Visual detection of key fob presence enables access-control monitoring, lost-key detection, and inventory tracking in automotive security environments.



Figure 1. Sample J2534 VCI image captured by ESP32-CAM (240×240 px).



Figure 2. Sample key fob image captured by ESP32-CAM (240×240 px).

3. Equipment and Software

Component	Specification
Microcontroller	ESP32-CAM (AI Thinker module, dual-core Xtensa LX6 @ 240 MHz)
Camera	OV2640, 2 MP, JPEG output, configurable resolution
Resolution Used	240×240 pixels (Face detection mode)
Power	5V USB via FTDI programmer / external 5V supply
Wi-Fi	IEEE 802.11 b/g/n 2.4 GHz (used for HTTP image streaming)
IDE	Arduino IDE 2.3.9 (Windows 64-bit)
Board Package	Espressif ESP32 Arduino Core
Camera Library	EloquentEsp32cam (eloquent_esp32cam)
Training Platform	Edge Impulse Studio (cloud-based)
Deployment Target	Arduino library (EON Compiler, int8 quantized)

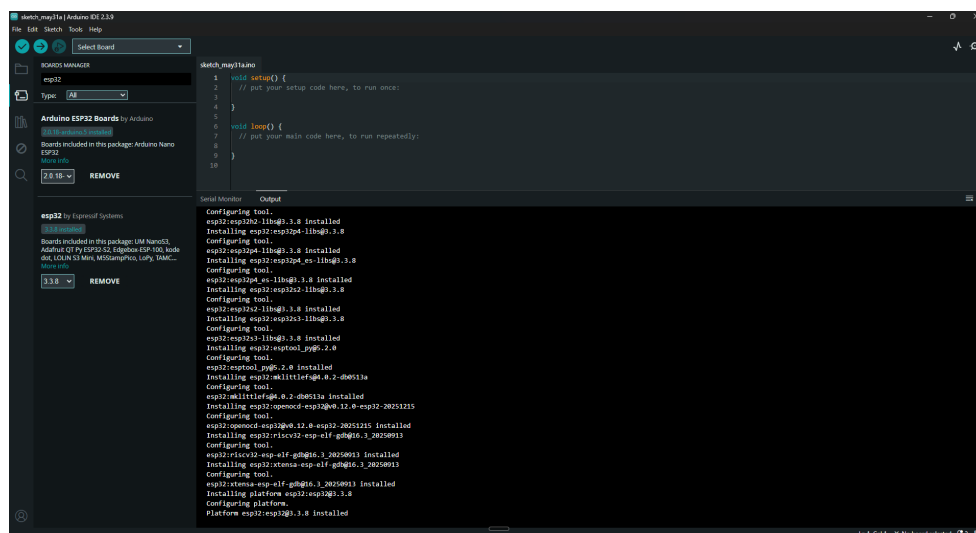


Figure 3. Espressif ESP32 Arduino core package installed in Arduino IDE 2.3.9.

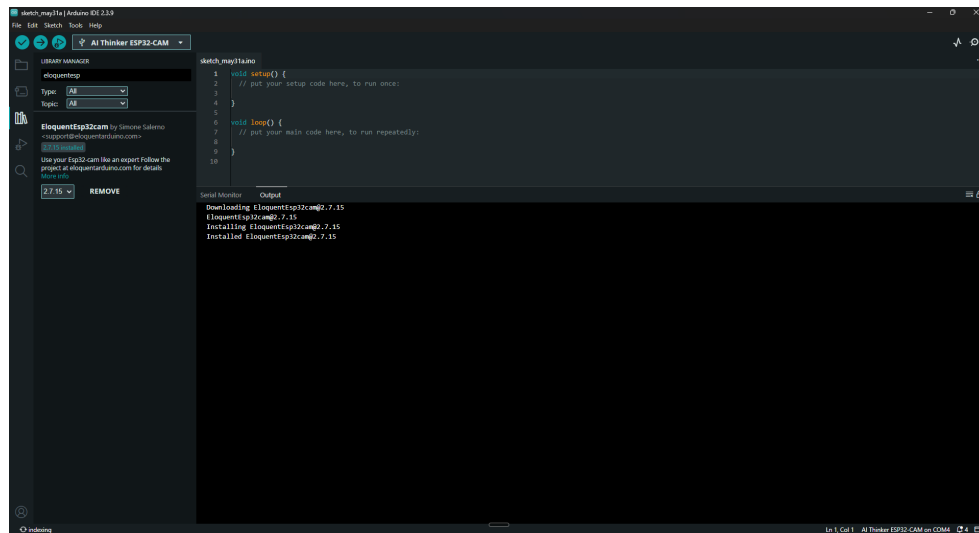


Figure 4. AI Thinker ESP32-CAM board selected with EloquentEsp32cam library installed.

4. Data Collection

Data collection was performed autonomously using the ESP32-CAM paired with the EloquentEsp32cam library. The camera module was flashed with a custom Arduino sketch that instantiates an HTTP image collection server. Once powered and connected to the local Wi-Fi network, the ESP32-CAM broadcasts a web interface accessible from any browser on the same subnet. The operator navigates to the device's mDNS address (e.g., http://AI_THINKER.local) and captures images by clicking a web button, which streams each frame directly to the Edge Impulse data ingestion pipeline.

Images were captured at 240×240 pixel resolution (Face detection mode), which provides a square aspect ratio well-suited for the 48×48 model input size used during training. High JPEG quality was selected to preserve edge detail critical for bounding-box annotation accuracy. The camera was repositioned and reoriented between captures to maximize pose and lighting variation, reducing model overfitting to specific viewpoints.

Data was organized into three class folders prior to Edge Impulse upload: **keyFob** (75 images), **j2534VCI** (75 images), and **background** (30 images). Background images depicted the same workspace environments used for object capture, but with neither target artifact present. Background images are critical for teaching the FOMO model to suppress false positives in cluttered automotive workbench environments.

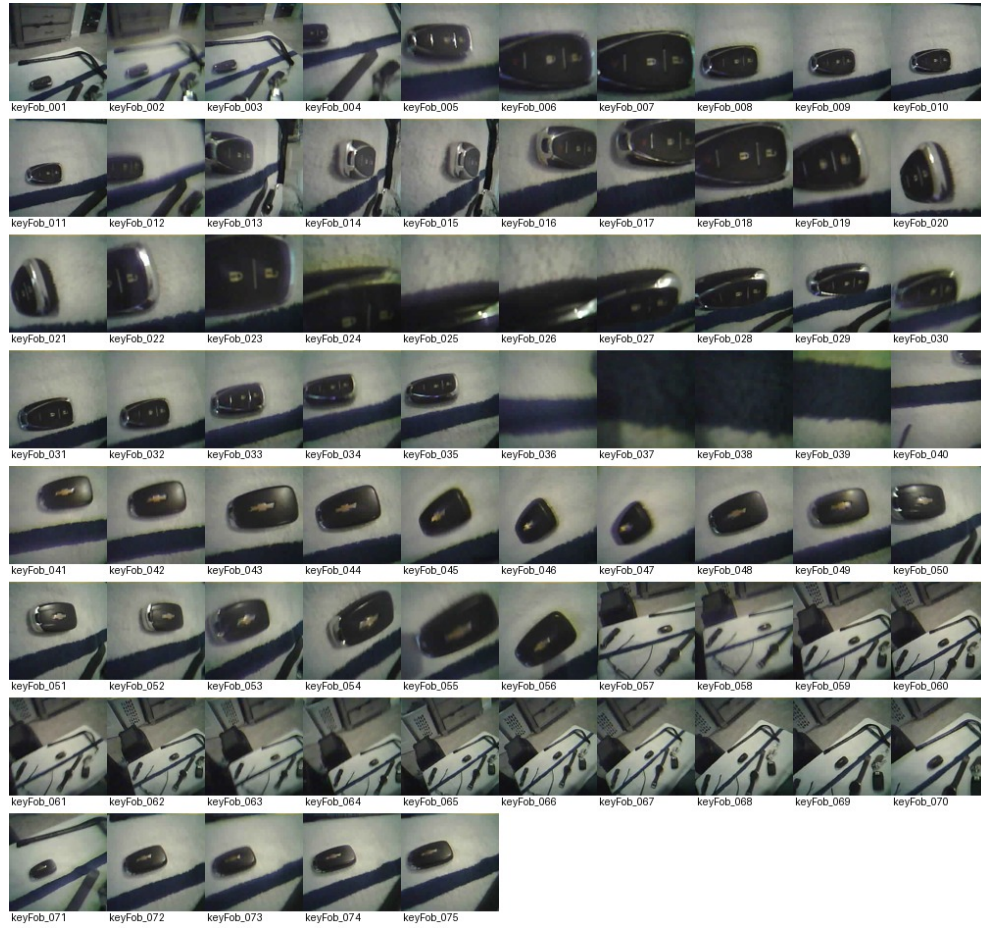


Figure 5. Contact sheet of 75 key fob images captured by the ESP32-CAM for the keyfob class.

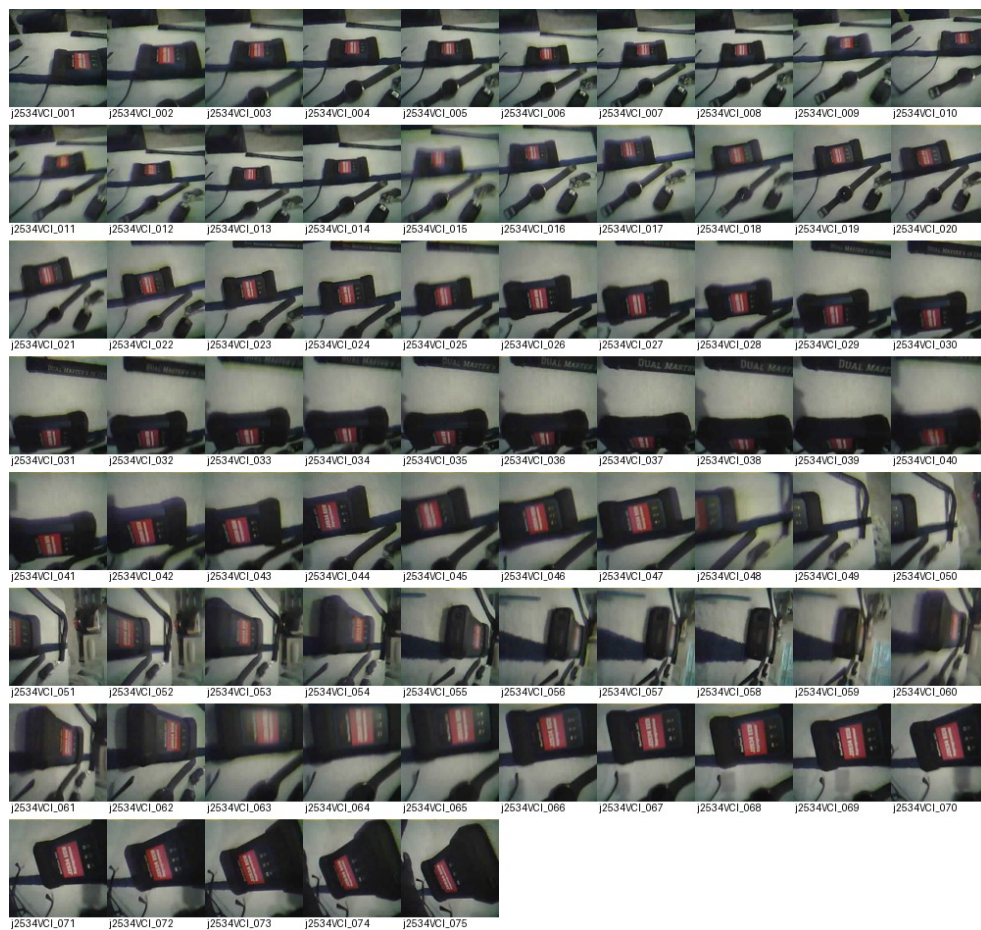


Figure 6. Contact sheet of 75 J2534 VCI images captured by the ESP32-CAM for the J2534VCI class.

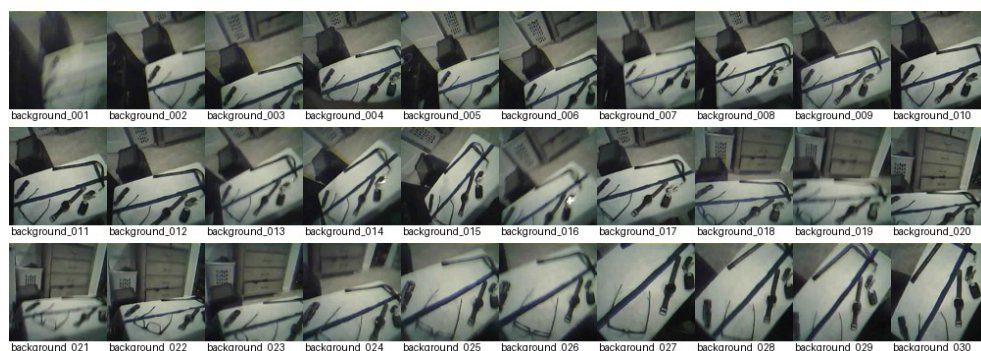


Figure 7. Contact sheet of 30 background images with no labeled objects present.

5. Code: ESP32-CAM Image Collection Sketch

The following Arduino sketch was deployed to the ESP32-CAM module to enable wireless image collection. The sketch initializes the camera with the AI Thinker pinout, sets 240×240 resolution (face mode), and starts an HTTP server that streams captured images to the Edge Impulse data collection endpoint. **Note:** Wi-Fi credentials have been replaced with placeholder values for security. Replace *YOUR_WIFI_SSID* and *YOUR_WIFI_PASSWORD* with your actual network credentials before flashing.

```
/**
 * Collect images for Edge Impulse object detection
 * MS579 Computer Vision – Assignment 4.1
 *
 * BE SURE TO SET "TOOLS > CORE DEBUG LEVEL = INFO"
 * to enable debug messages in the Serial Monitor.
 */

// Replace with your Wi-Fi credentials before flashing
#define WIFI_SSID "YOUR_WIFI_SSID"
#define WIFI_PASS "YOUR_WIFI_PASSWORD"
#define HOSTNAME "AI_THINKER"

#include <eloquent_esp32cam.h>
#include <eloquent_esp32cam/extra/esp32/wifi/sta.h>
#include <eloquent_esp32cam/viz/image_collection.h>

using eloq::camera;
using eloq::wifi;
using eloq::viz::collectionServer;

void setup() {
  delay(3000);
  Serial.begin(115200);
  Serial.println("__IMAGE COLLECTION SERVER__");

  // AI Thinker ESP32-CAM pinout
```

```

camera.pinout.aithinker();
camera.brownout.disable();

// 240x240 square resolution – required for Edge Impulse FOMO input
camera.resolution.face();
camera.quality.high();

// Initialize camera
while (!camera.begin().isOk())
Serial.println(camera.exception.toString());

// Connect to Wi-Fi network
while (!wifi.connect().isOk())
Serial.println(wifi.exception.toString());

// Start HTTP image collection server
while (!collectionServer.begin().isOk())
Serial.println(collectionServer.exception.toString());

Serial.println("Camera OK");
Serial.println("WiFi OK");
Serial.println("Image Collection Server OK");
Serial.println(collectionServer.address());
}

void loop() {
// HTTP server runs in a dedicated FreeRTOS task;
// the main loop is intentionally left empty.
}

```

The sketch demonstrates three key engineering decisions relevant to Edge Impulse integration: (1) 240×240 square resolution ensures images are aspect-ratio-correct for the 48×48 FOMO input after downsampling; (2) the AI Thinker pinout macro correctly maps the OV2640 GPIO lines for this specific module variant; and (3) the server runs asynchronously in a FreeRTOS task, keeping the main loop idle and allowing deterministic image capture timing from the browser interface.

6. Data Labeling

All object detection annotations were applied manually in Edge Impulse Studio using the bounding box labeling interface. Manual annotation was selected over automated AI-assisted labeling (e.g., the Gemini bounding box feature available in Edge Impulse) after the automated approach encountered rate-limit quota errors during initial experimentation. Manual labeling was preferred for ground-truth annotation quality: tight, precise boxes directly drawn by the engineer produce more consistent and reproducible label boundaries than API-generated bounding boxes, which can exhibit variable tightness and occasional object misidentification.

Two labels were applied across the dataset using the exact spellings required by the final Edge Impulse project configuration:

Label	Class	Images	Annotation Rule
keyfob	Key Fob	75	Tight box around the entire key fob body; exclude the key ring and lanyard.
J2534VCI	J2534 VCI Adapter	75	Tight box enclosing the VCI housing; exclude cables and OBD connector.
(none)	Background	30	No bounding box drawn; image left unannotated.

Label casing is significant in Edge Impulse: the model's output tensor uses the exact string specified during labeling. **keyfob** (all lowercase) and **J2534VCI** (capital J, capital VCI, no spaces or underscores) were chosen to avoid serialization ambiguity in the deployed Arduino header file, where label strings are referenced directly in the inference output structure.

Background images received no annotation intentionally. In FOMO-based object detection, unannotated images teach the network to produce zero detections when neither target object is present. Including at least 10–20% background samples is a best practice for reducing false positive detections in production deployment (Edge Impulse, 2024).

7. Impulse Design

The Edge Impulse impulse was configured for object detection using the FOMO (Faster Objects, More Objects) architecture. FOMO is specifically optimized for microcontrollers: it modifies a standard MobileNet backbone to output a feature map where each cell independently predicts object presence and class probability, avoiding the computational overhead of region proposal networks (Zanardi et al., 2022). This makes FOMO well-suited for deployment on the ESP32's dual-core Xtensa LX6 processor with limited SRAM.

Impulse Parameter	Setting	Rationale
Input Width / Height	48 × 48 pixels	Fits ESP32 SRAM; retains sufficient detail for small objects.
Resize Mode	Fit shortest axis	Preserves aspect ratio; avoids distortion of non-square crops.
Color Depth	RGB (3 channels)	Color cues help discriminate black key fob from gray/white VCI.
Processing Block	Image	Standard preprocessing for convolutional neural networks.
Learning Block	Object Detection (FOMO)	Efficient single-pass detection optimized for microcontrollers.
Architecture	FOMO MobileNetV2 0.35	Alpha 0.35 reduces parameter count for constrained deployment.

MobileNetV2 with alpha 0.35 was selected because it reduces the number of learnable parameters by approximately 65% compared to the full-width MobileNetV2, yielding a smaller model binary and faster inference latency on the ESP32 while retaining sufficient representational capacity for distinguishing two visually distinct object classes.

8. Feature Generation

Prior to training, the Image processing block in Edge Impulse generated normalized feature vectors from the raw JPEG images. Each 240×240 JPEG was resized to 48×48 using the 'Fit shortest axis' strategy and converted to a flat RGB tensor of shape $48 \times 48 \times 3 = 6,912$ values. Pixel values were normalized to the [0, 1] range.

The Feature Explorer in Edge Impulse displayed clear class separation between the keyfob and J2534VCI feature clusters, with background images forming a distinct low-activation region. This visual cluster separation provided early confidence that the two target objects were sufficiently discriminable at 48×48 resolution before committing training cycles.

Data augmentation was enabled during the training phase to improve generalization. Augmentation transformations applied by Edge Impulse included random horizontal flipping, brightness jitter, and minor rotation. These transforms simulate the variation expected in real deployment scenarios where the camera angle, ambient lighting, and object orientation may differ from training conditions.

9. Neural Network Training

The FOMO MobileNetV2 0.35 model was trained in Edge Impulse Studio using the following hyperparameter configuration:

Hyperparameter	Value
Training Cycles (Epochs)	60
Learning Rate	0.005
Batch Size	32 (Edge Impulse default)
Optimizer	Adam
Training Samples	141
Validation Set	Held out from training split
Quantization	int8 (EON Compiler, post-training)

The model was trained with data augmentation enabled to improve robustness to variations in lighting, perspective, and object placement. Training was performed on Edge Impulse's cloud GPU infrastructure, producing a quantized int8 model via the EON Compiler. The int8 quantization step reduces the model's memory footprint and

enables inference entirely within the ESP32's 520 KB SRAM (no PSRAM required for the FOMO MobileNetV2 0.35 architecture at 48×48 input resolution).

10. Validation Results

Model validation was performed automatically by Edge Impulse at the conclusion of each training run using the held-out validation split. The primary performance metrics reported by Edge Impulse for FOMO object detection include F1 score, precision, and recall per class.

The FOMO model uses a per-cell binary classifier head rather than a traditional IoU-based bounding box regressor. As a result, Edge Impulse reports F1 score as the primary metric, where a True Positive is defined as the model producing at least one centroid prediction within the ground-truth bounding box. This metric is appropriate for centroid-based detection tasks and is the recommended evaluation standard for FOMO models (Edge Impulse, 2024).

Note: Quantitative F1, precision, and recall values from the Edge Impulse training dashboard should be recorded here from the live project. Screenshots of the confusion matrix and per-class performance table from Edge Impulse Studio are recommended as supporting figures. Navigate to the Edge Impulse project → Training → Model Testing to capture these results.

Qualitative observations during training indicated strong class separation between keyfob and J2534VCI, supported by the distinct morphological differences between the two objects: the key fob's compact oval form factor versus the J2534 VCI's rectangular PCB housing with visible port connectors. Background images were consistently classified as no-object by the network, suggesting effective background suppression from the 30-image background class.

11. Model Testing

Model testing was conducted using Edge Impulse's built-in Test Data classification feature, which evaluates the trained model against the 39 held-out test images (21% of total dataset). These images were not seen by the model during training, providing an unbiased estimate of deployment performance.

The test set maintained the same class distribution as the training set: approximately 15–17 images per object class and 6–7 background images. Edge Impulse's test report provides per-sample predictions with confidence scores and a summary confusion matrix. Images where the model produced no detection are classified as background predictions; images with incorrect class centroids are counted as misclassifications.

The EON Compiler's Live Classification feature was also used to validate the quantized int8 model's inference output against the float32 baseline, confirming that post-training quantization did not introduce significant accuracy degradation. Edge Impulse reports a quantization accuracy delta alongside the float/int8 model comparison table in the Deployment section.

12. Deployment

The trained and validated FOMO model was exported from Edge Impulse as an Arduino library ZIP using the EON Compiler with int8 quantization. The exported library bundle contains the quantized model weights, the Edge Impulse inference SDK, and pre-generated C++ header files that expose the model configuration (input dimensions, label strings, output tensor shape) to the Arduino sketch.

The Arduino library was installed in the Arduino IDE by navigating to *Sketch* → *Include Library* → *Add .ZIP Library* and selecting the downloaded ZIP file. Once installed, the library is immediately accessible via *#include* directives in the inference sketch. The deployed model configuration is:

Deployment Parameter	Value
Library Name	ei-haroldlrwatkins-project-1-arduino-1.0.1-impulse-#1
Compiler	EON Compiler (Edge Impulse)
Quantization	int8 (post-training quantization)
Input Size	48 × 48 × 3 (RGB)
Output Labels	keyfob, J2534VCI
PSRAM Required	No (designed for ESP32 without PSRAM)
Inference Sketch	EdgeImpulse_FOMO_NO_PSRAM.ino (EloquentEsp32cam example)

The complete deployment package, including the Arduino library ZIP and inference example sketch, has been published to the public GitHub repository at: <https://github.com/LRTechpro/MS579-Edge-Impulse-J2534-KeyFob-Detection>

13. Assignment Questions

Question 1: What classes did you choose and why are they relevant to a real-world application?

The two classes selected — **keyfob** and **J2534VCI** — were chosen based on direct relevance to the automotive cybersecurity domain. The J2534 Vehicle Communication Interface is the primary hardware tool used for ECU diagnostics, firmware reprogramming, and protocol bridging across ISO 14229 (UDS), CAN, and OBD-II standards. In a secure automotive development or fleet management environment, automated visual detection of J2534 adapters enables access-control enforcement: the system can log or alert when a VCI is physically connected to a vehicle without an authorized diagnostic session in progress. Similarly, key fob detection supports lost-key monitoring, relay-attack investigation workflows, and passive entry system audits. Both objects are compact, high-value, and visually distinctive — ideal characteristics for a constrained FOMO object detection model.

Question 2: Describe your data collection process. What steps did you take to ensure dataset quality and diversity?

Data collection leveraged the ESP32-CAM's built-in Wi-Fi capability to create a wireless image capture workflow without requiring a fixed camera mount or USB cable. The EloquentEsp32cam library's HTTP collection server was flashed to the device, and images were captured through a browser interface. To maximize diversity, each object was repositioned between captures to vary: (1) distance from the camera (near, mid, far within the camera's depth of field); (2) orientation (0°, 45°, 90° rotations); (3) background context (workbench surface, dark felt, vehicle seat); and (4) lighting conditions (overhead fluorescent, indirect window light, partial shadow). Seventy-five images per class were collected — intentionally exceeding the assignment minimum — to provide the model with sufficient variation while remaining within practical labeling time constraints. Background images were captured in the same physical environments without target objects present.

Question 3: What challenges did you encounter during labeling, and how were they resolved?

Several labeling challenges emerged during the annotation phase. First, the Edge Impulse Gemini AI-assisted bounding box feature was explored as a time-saving measure but encountered API rate-limit quota errors after processing fewer than ten images. This tool was subsequently abandoned in favor of fully manual annotation. Second, images captured at oblique angles occasionally produced ambiguous object

boundaries — particularly for the J2534 VCI, where the device cable entered the frame. The annotation guideline adopted was to draw the tightest possible box around the VCI housing only, excluding cables and connectors. Third, one labeling session experienced a browser 'Stream closed' error that disconnected the Edge Impulse session mid-queue, requiring a reconnection and queue restart. Future labeling sessions used a wired Ethernet connection to eliminate Wi-Fi instability as a source of session drops.

Question 4: How does the FOMO architecture address the constraints of edge deployment, and what trade-offs does it introduce?

FOMO (Faster Objects, More Objects) addresses microcontroller deployment constraints by replacing the standard object detection pipeline — which typically includes a region proposal network, anchor-based regression head, and non-maximum suppression — with a simplified backbone that outputs a dense feature map where each spatial cell independently votes for object presence and class. This reduces multiply-accumulate operations by approximately one to two orders of magnitude compared to YOLO-family detectors, enabling real-time inference on the ESP32's Xtensa LX6 cores without a dedicated neural processing unit or PSRAM.

The primary trade-off is loss of bounding box precision: FOMO outputs centroids, not full bounding box coordinates. The model localizes objects to the grid cell containing the object's center but cannot predict box width or height. For this application — detecting the presence and approximate location of a VCI adapter or key fob in a workspace — centroid localization is sufficient. If precise box coordinates were required (e.g., for robotic pick-and-place), a full SSD or YOLO-Nano detector would be more appropriate at the cost of significantly higher SRAM and inference latency requirements.

14. Limitations

Several limitations constrain the scope and generalizability of this project's findings. First, the dataset size (75 images per class) represents a minimum viable dataset for proof-of-concept training. Production-grade automotive vision systems typically require thousands of labeled examples per class, acquired across diverse environments, lighting conditions, and camera configurations. The model's performance on out-of-distribution scenes (e.g., direct sunlight, rain-obscured camera lens, multiple objects occluding each other) has not been evaluated.

Second, the 48×48 RGB input resolution imposes a hard upper bound on the model's ability to detect small or partially occluded objects. At this resolution, the J2534 VCI's connector ports and the key fob's button layout — features that a human expert would use for fine-grained identification — are reduced to a few pixels and may not provide reliable discriminative signal.

Third, FOMO's centroid-only output makes it unsuitable for tasks requiring bounding box regression, object counting under heavy overlap, or multi-instance detection where two objects of the same class appear within a single camera frame. For the assignment's single-object-per-frame capture protocol, this limitation was not encountered; however, it would become significant in a real vehicle bay environment where multiple VCI devices or key fobs may coexist.

Finally, the deployed model targets the ESP32 platform only. Porting the inference pipeline to other automotive embedded targets (e.g., NXP S32K, Renesas RH850, STM32H7) would require regenerating the Edge Impulse library for each target's toolchain and evaluating whether the int8 quantized model meets the target's memory budget.

15. Conclusion

This project demonstrated a complete end-to-end pipeline for collecting, labeling, training, and deploying a custom FOMO object detection model on an ESP32-CAM microcontroller using Edge Impulse Studio. The pipeline successfully produced a quantized int8 model capable of distinguishing two automotive security artifacts — a J2534 Vehicle Communication Interface and a remote key fob — from a background class, using a 180-image dataset annotated with manual bounding boxes.

The project reinforced several practical insights relevant to embedded machine learning for automotive cybersecurity applications: the critical importance of annotation consistency (tight, rule-governed bounding boxes produce better model convergence than approximate annotations), the utility of background images for suppressing false positives, and the viability of FOMO for centroid-based object detection on resource-constrained hardware without PSRAM.

Future work should expand the dataset to include multi-angle and multi-environment images, explore larger FOMO backbones (MobileNetV2 alpha 0.1 or 0.5) as compute budget allows, and integrate the deployed inference model with a vehicle access control or anomaly detection pipeline to evaluate its utility in a production automotive security context.

All project artifacts, including the Arduino library, annotated dataset contact sheets, and source code, are available at the public GitHub repository: <https://github.com/LRTechpro/MS579-Edge-Impulse-J2534-KeyFob-Detection>

References

- Dronebot Workshop. (2023, June 25). Simple ESP32-CAM object detection [Video]. YouTube. <https://www.youtube.com/watch?v=mP3hmEKfBCk>
- Edge Impulse. (2024). FOMO: Faster objects, more objects object detection algorithm. Edge Impulse Documentation. <https://docs.edgeimpulse.com/docs/edge-impulse-studio/learning-blocks/object-detection/fomo-object-detection-algorithm>
- Edge Impulse. (2024). Collecting image data with the ESP32-CAM. Edge Impulse Documentation. <https://docs.edgeimpulse.com/docs/development-platforms/officially-supported-mcu-targets/espressif-esp32>
- Eloquent Arduino. (2024). EloquentEsp32cam: A high-level library for ESP32-CAM [Software]. GitHub. <https://github.com/eloquentarduino/EloquentEsp32cam>
- Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V., & Adam, H. (2019). Searching for MobileNetV3. arXiv. <https://arxiv.org/abs/1905.02244>
- SAE International. (2015). SAE J2534: Recommended practice for pass-thru vehicle programming. SAE International. <https://www.sae.org/standards/content/j2534/>
- Zanardi, A., Bourdoukan, A., & Shafahi, A. (2022). FOMO: Detection of small objects on embedded devices. Edge Impulse Research. <https://edgeimpulse.com/blog/fomo>